WorksheetDownload.com

10 CONCEPTUAL Q&A DEVOPS · PRACTICE WORKSHEET

CONCEPTUAL SERIES

AWS CloudFormation

Name: ________ Cohort/Batch: ________ Date: ________ Score: ________

AWS CLOUDFORMATION PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A DevOps

TOTAL: 10 QUESTIONS

Q1 What problem does AWS CloudFormation solve in DevOps or infrastructure? *Threat Model*

Q2 What is AWS CloudFormation's core architecture? *Architecture*

Q3 How does AWS CloudFormation define infrastructure or configuration as code? *Configuration*

Q4 How does AWS CloudFormation ensure idempotency? *Hardening*

Q5 How do you handle secrets in AWS CloudFormation? *SSO / IdP*

Q6 How does AWS CloudFormation integrate with CI/CD pipelines? *Observability*

Q7 How do you debug a failed AWS CloudFormation deployment? *Lifecycle*

Q8 What are security best practices for AWS CloudFormation? *Compliance*

Q9 How does AWS CloudFormation scale for large environments? *Testing*

Q10 What are common mistakes teams make when adopting AWS CloudFormation? *Tuning*

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 AWS CloudFormation addresses a specific — Mitigation

AWS CloudFormation addresses a specific concern provisioning, configuration management, orchestration, or CI/CD by automating manual steps that are error-prone, slow, or not reproducible when done by hand.

A2 AWS CloudFormation has a control plane — Primitives

AWS CloudFormation has a control plane that stores desired state and a data plane (agents or push-based SSH) that enforces it. Understanding this distinction clarifies where configuration is evaluated and where changes are applied.

A3 AWS CloudFormation uses declarative files — Hardening

AWS CloudFormation uses declarative files (YAML, HCL, or a DSL) to express desired state. A plan/diff phase shows what will change before applying. Version-controlling these files enables peer review and rollback.

A4 AWS CloudFormation operations are designed to — Gotchas

AWS CloudFormation operations are designed to produce the same result when run multiple times. Applying the same config twice converges to the desired state rather than creating duplicate resources. This makes automation safe to re-run.

A5 Secrets should never be stored in — Federation

Secrets should never be stored in plain text in AWS CloudFormation config files. Use a secrets backend (Vault, AWS Secrets Manager) and inject values at runtime via environment variables or encrypted vaults supported by the tool.

A6 CI/CD pipelines call AWS CloudFormation in — Observability

CI/CD pipelines call AWS CloudFormation in automated steps: plan on a pull request to preview changes and apply on merge to main. Approval gates can require a human to review the plan before changes go to production.

A7 Check AWS CloudFormation's logs for the — Automation

Check AWS CloudFormation's logs for the specific error message and the resource that failed. For infrastructure tools, re-run with increased verbosity (-v, --debug). Review the state file or event history to understand what changed before the failure.

A8 Rotate credentials used by AWS — Governance

Rotate credentials used by AWS CloudFormation, restrict network access to its control plane, enable audit logging of all operations, and use least-privilege service accounts. Never run AWS CloudFormation with admin credentials in production.

A9 Large AWS CloudFormation deployments use — Assessment

Large AWS CloudFormation deployments use workspaces or namespaces to isolate environments, remote state backends for team collaboration, and modular code to avoid a single monolithic config that is slow to plan/apply.

A10 Common pitfalls include storing secrets in — Optimization

Common pitfalls include storing secrets in plain text config, skipping the plan step before apply, not reviewing state drift, and not having a rollback strategy when a deployment fails partway through.